

LLM-Assisted I/O Bottleneck Diagnosis in HPC Workflows

CS546 Term Project Report

Alistair Dreux–Egger
adreuxegger@hawk.illinoistech.edu

Jiyi Chen
jchen212@hawk.illinoistech.edu

Meng Tang (PhD Lead)
mtang11@hawk.illinoistech.edu

Illinois Institute of Technology
Chicago, IL 60616, USA

Abstract

I/O bottleneck diagnosis in high-performance computing workflows is challenging because root causes depend on interacting signals from workflow structure, task parallelism, storage behavior, and resource utilization. This project investigates whether large language models can assist diagnosis from structured workflow execution snapshots. We construct a workflow-informed synthetic benchmark of 500 workflow snapshots covering 11 bottleneck categories, and implement a reproducible pipeline comparing five strategies: a rule-based baseline, classify-only prompting, zero-shot prompting, few-shot prompting, and chain-of-thought prompting. Experiments with GPT-4.1-mini and Claude Haiku 4.5 show that domain-informed prompting substantially improves diagnostic accuracy over low-context prompting, while zero-shot, few-shot, and chain-of-thought strategies present different accuracy-cost tradeoffs. These findings suggest that LLMs can serve as useful assistants for structured workflow I/O analysis.

Index Terms

HPC workflows, I/O bottleneck diagnosis, large language models, prompt engineering.

I. INTRODUCTION

Modern scientific and data-intensive applications are increasingly executed as complex workflows composed of multiple computational stages and data movement operations [1, 2]. In such environments, I/O performance often plays a dominant role in determining end-to-end execution time [3]. However, diagnosing I/O bottlenecks from observed metrics remains challenging because performance issues typically arise from interactions across workflow stages and system components. A workflow consists of interdependent stages with complex dataflow relationships, where tasks produce and consume data through shared storage systems. As a result, bottlenecks are rarely localized to a single metric, but instead emerge from system-level interactions such as concurrent access, data movement, and resource contention. Moreover, similar

observable symptoms, such as high I/O time or low CPU utilization, may correspond to different underlying causes depending on access patterns and storage configurations. Effective diagnosis therefore requires integrating heterogeneous signals and reasoning about their combined effects, rather than relying on isolated metrics or simple threshold-based rules.

Existing HPC monitoring and analysis tools provide detailed visibility into runtime behavior, including I/O throughput, latency, access patterns, and execution timelines (cite: HPC I/O profiling tools such as Darshan, Recorder, or tracing systems). While these tools are effective for collecting and summarizing system-level metrics, they do not directly provide root-cause diagnoses of performance issues. In practice, users must interpret multiple signals, correlate them across workflow stages, and relate them to potential bottleneck causes based on domain knowledge (cite: workflow performance analysis / I/O characterization studies). As a result, bridging the gap between low-level measurements and high-level diagnosis remains difficult, particularly in complex workflows where multiple factors interact and simple heuristics may not generalize across applications.

Recent advances in large language models (LLMs) suggest a potential opportunity to bridge this gap [4, 5]. LLMs have demonstrated strong capabilities in reasoning over structured and semi-structured inputs, integrating information across multiple signals, and generating human-interpretable outputs [6]. These properties make LLMs a promising candidate for diagnostic tasks that require combining heterogeneous signals and inferring underlying causes, particularly in settings where explicit reasoning is needed to connect low-level measurements to high-level diagnoses.

In this work, we systematically evaluate the feasibility of using LLMs to diagnose I/O bottlenecks in workflow executions. Given structured snapshots of workflow execution data, the goal is to evaluate whether LLMs can accurately identify bottleneck categories and generate structured diagnostic outputs that summarize the key evidence used for the prediction. The study focuses on dataset curation, prompt design, diagnostic accuracy, cost-efficiency, latency, and analysis of common misclassification patterns.

II. BACKGROUND

Workflow performance analysis has traditionally relied on monitoring, profiling, and tracing tools [7], as well as automated or machine-learning-based methods [8], to expose low-level system metrics such as I/O throughput, latency, access patterns, and execution timelines. While these approaches are effective at capturing and analyzing runtime behavior, translating such signals into accurate, workflow-level diagnosis

of I/O bottlenecks typically requires substantial domain expertise. Most existing methods emphasize measurement, detection, or prediction, and provide limited support for explicit, human-interpretable diagnostic reasoning, particularly in complex workflows where multiple factors interact.

In workflow-based HPC applications, I/O behavior is shaped by the structure of the workflow and the dataflow relationships between stages. A workflow typically consists of multiple stages with task-level parallelism, where data is produced and consumed through shared storage systems. Common dataflow patterns, such as one-to-one, one-to-many, and many-to-one communication, can lead to different I/O access characteristics, including bursty traffic, uneven data distribution, and contention on shared resources (cite: workflow dataflow / producer-consumer / workflow I/O characterization). Bottlenecks therefore often arise from interactions between tasks, storage systems, and data movement patterns, rather than from a single isolated metric or component. Understanding these interactions is essential for diagnosing workflow-level I/O performance issues.

More recently, large language models (LLMs) have been investigated for systems-related tasks such as log analysis [9], debugging assistance [10], and performance interpretation. Their ability to reason over structured inputs and generate natural-language explanations suggests potential for supporting diagnostic workflows. However, the applicability of LLMs to structured workflow I/O bottleneck diagnosis has not been well examined, especially in settings where the model must map multiple workflow and I/O signals to a specific bottleneck category.

In particular, LLMs can be viewed as a flexible reasoning layer that maps structured inputs to diagnostic decisions by leveraging implicit knowledge learned during pretraining. This capability aligns with the needs of workflow I/O diagnosis, where multiple heterogeneous signals must be integrated to infer an underlying cause. However, it remains unclear how reliably LLMs can perform such structured diagnostic reasoning under controlled conditions.

III. PROBLEM FORMULATION

A. *I/O Bottleneck Diagnosis as a Reasoning Task*

I/O bottleneck diagnosis in high-performance computing (HPC) workflows aims to identify the dominant factors that limit application performance based on observed execution behavior. In practice, such diagnosis is typically performed by domain experts who analyze multiple interacting signals, including execution statistics, I/O characteristics, and system-level metrics, and reason about their combined effects.

In this work, we formulate I/O bottleneck diagnosis as a structured reasoning task over workflow execution snapshots, as illustrated in Figure 1. Each snapshot $x \in \mathcal{X}$ is represented as a structured collection of features that summarize the behavior of a workflow stage, including execution statistics, I/O-related metrics, and data dependencies. Given such a snapshot, the goal is to infer the underlying bottleneck by integrating multiple sources of evidence.

Unlike standard classification problems that rely on isolated features, accurate diagnosis requires reasoning over heterogeneous signals and identifying how they jointly indicate a performance issue. Therefore, we model the task as a mapping

$$f(x) \rightarrow (y, e)$$

where $y \in Y$ denotes the predicted bottleneck category and e denotes structured supporting evidence (e.g. key signals and a short explanation). In this project, the quantitative evaluation focuses on the correctness of y , while e is used for structured reporting and is not separately scored.

This formulation reflects the interpretive nature of I/O bottleneck analysis, where a useful diagnostic output should identify a bottleneck category and expose the main evidence behind the prediction.

B. Research Questions

Based on the formulation of I/O bottleneck diagnosis as a structured reasoning task, we aim to systematically evaluate the capability of large language models (LLMs) for this problem. Our investigation follows a progressive logic from feasibility to optimization and practical deployment.

We first examine whether LLMs can accurately perform bottleneck diagnosis from structured workflow snapshots, as this determines whether the proposed formulation is viable compared to traditional expert-driven approaches. Given that LLM performance is highly sensitive to input design, we then investigate how different prompting strategies, which provide varying levels of domain knowledge and reasoning

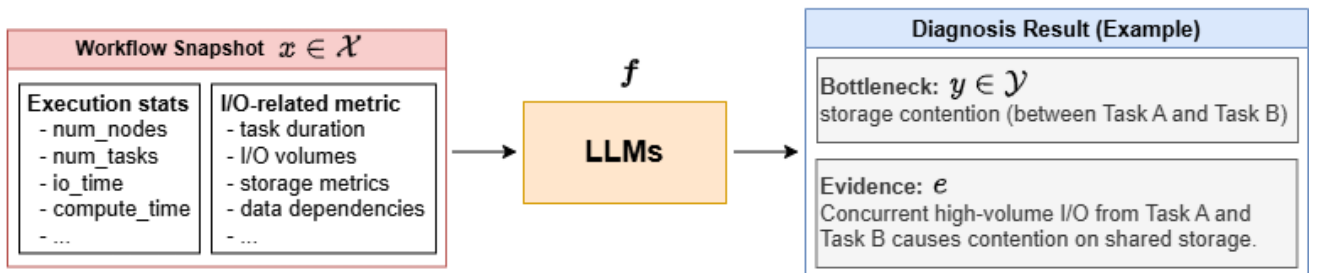


Fig. 1. I/O bottleneck diagnosis formulated as a structured reasoning task over workflow snapshots.

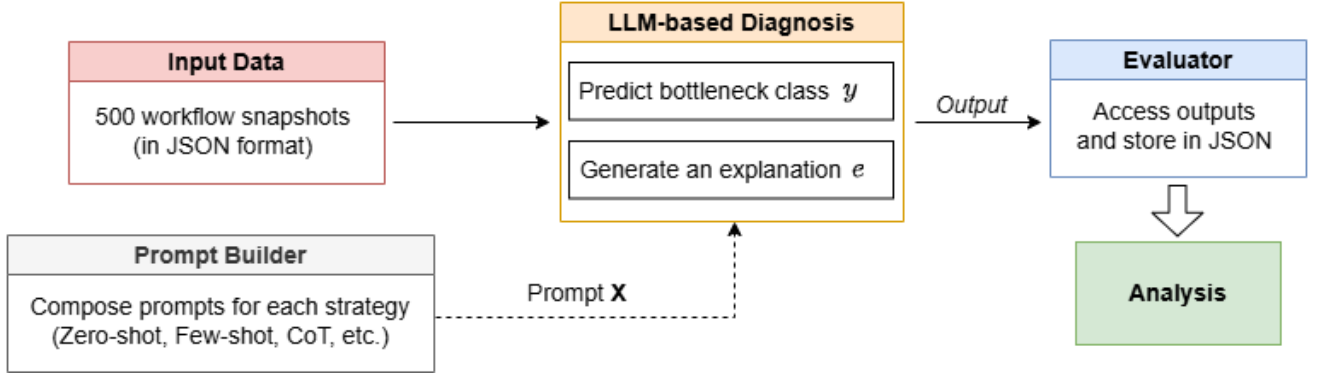


Fig. 2. Overview of the LLM-based I/O bottleneck diagnosis pipeline.

guidance, influence diagnostic effectiveness. Finally, even if high accuracy can be achieved, practical deployment requires careful consideration of computational cost, since LLM-based methods incur non-trivial token usage and latency. Therefore, it is necessary to understand the trade-off between diagnostic performance and computational efficiency across different strategies.

Formally, we study the following research questions:

- **RQ1 (Accuracy).** To what extent can large language models accurately diagnose I/O bottlenecks from structured workflow snapshots?
- **RQ2 (Prompting Strategies).** How do different prompting strategies affect the diagnostic performance of large language models on this task?
- **RQ3 (Cost–Accuracy Tradeoff).** What trade-offs exist between diagnostic accuracy and computational cost, in terms of token usage and latency, across different strategies?

Together, these questions define the scope of our study and guide the design of the dataset, prompting strategies, evaluation metrics, and experimental analysis presented in the following sections.

IV. METHODOLOGY

A. Diagnosis Pipeline

To systematically evaluate LLM-based I/O bottleneck diagnosis, we design a unified diagnosis pipeline that processes workflow snapshots, applies different prompting strategies, and evaluates the resulting predictions. The overall workflow is illustrated in Figure 2.

The pipeline takes as input structured workflow snapshots that summarize execution behavior, including I/O characteristics, resource utilization, and system-level signals. For each snapshot, a prompt is constructed according to a specified strategy, which determines how domain knowledge and reasoning

guidance are presented to the language model. The prompt is then submitted to the LLM, which generates a diagnosis consisting of a predicted bottleneck category and, for most prompting strategies, structured supporting evidence such as confidence, key signals, and a concise explanation.

To enable fair comparison across strategies, all prompting methods are applied to the same set of snapshots in a controlled and sequential manner. This ensures that performance differences arise solely from the prompting design rather than variations in data or execution conditions. For each model invocation, the pipeline records detailed runtime statistics, including prompt and completion token usage, API latency, and system resource consumption.

The outputs generated by the LLM are parsed into a structured format and validated against a predefined schema to ensure consistency. Responses that fail to meet the required format are treated as incorrect predictions. All valid outputs, along with their associated metadata, are stored for downstream evaluation and analysis.

The pipeline consists of several key components, including dataset construction, prompting strategies, and evaluation mechanisms, which are described in detail in the following subsections.

B. Dataset Curation

A key challenge in this project is that real HPC workflow traces with expert-verified I/O bottleneck labels are difficult to obtain at sufficient scale. Therefore, we construct a workflow-informed synthetic dataset. The dataset is synthetic in the sense that the numerical metric values are generated programmatically; however, its schema and bottleneck logic are designed based on workflow-level I/O analysis concepts, such as workflow stages, task-level execution behavior, storage metrics, and data dependencies. This design is inspired by the workflow analysis in Storage Performance Matching Analysis System [11], where workflow templates and dataflow structures are used to represent real scientific workflow behavior. In our dataset generation process, each snapshot is created by selecting a workflow context, assigning a bottleneck class, generating class-specific metric patterns, and attaching a ground-truth annotation containing the bottleneck label, confidence level, explanation, and key diagnostic signals.

The workflow contexts are chosen to cover a broad range of scientific and data processing domains, including genomics, astronomy, bioinformatics, seismology, climate, molecular dynamics, machine learning, and ETL workloads. Examples include workflow families such as `1000_genomes`, `montage`, `cybershake`, `namd`, `wrf`, and `spark_etl`. These workflow names provide workflow-inspired contexts in which controlled bottleneck signatures can be embedded. This allows us to evaluate whether LLMs can

reason over structured I/O-related signals while keeping the ground truth known and the class distribution controlled.

The final dataset contains 500 workflow snapshots covering 11 bottleneck classes. The class distribution is approximately balanced, as shown in Table I. This balance is important because it prevents the evaluation from being dominated by a few frequent bottleneck types and allows us to compare diagnostic performance across different classes.

TABLE I
DISTRIBUTION OF BOTTLENECK CLASSES IN THE 500-SNAPSHOT DATASET.

Class	Count	Percentage
read_bandwidth_saturation	52	10.4%
io_interference	50	10.0%
data_skew	50	10.0%
staging_inefficiency	50	10.0%
compute_bound	44	8.8%
metadata_contention	43	8.6%
serialized_io	43	8.6%
checkpointing_overhead	43	8.6%
lock_contention	42	8.4%
network_io_bottleneck	42	8.4%
storage_bandwidth_saturation	41	8.2%

Each snapshot is stored as a JSON object and organized into several semantic groups. This structured format makes the input easy for both programs and LLMs to parse, while still preserving the main signals needed for diagnosis. The structured fields are summarized below:

- **Workflow metadata:** workflow name, workflow domain, and number of stages.
- **Stage metadata:** stage name, stage order, predecessor stages, and operation type.
- **Execution statistics:** number of nodes, number of tasks, parallelism, total execution time, I/O time, compute time, and I/O time ratio.
- **I/O metrics:** read bandwidth, write bandwidth, peak storage bandwidth, bandwidth utilization ratio, IOPS, average I/O size, and sequential access ratio.
- **Resource utilization:** CPU utilization, memory utilization, network bandwidth, and network utilization ratio.
- **Storage configuration:** storage type, stripe count, and whether the storage system is shared.
- **Annotation:** ground-truth bottleneck label, confidence level, natural-language explanation, and key diagnostic signals.

The annotation field is particularly important because it provides the ground-truth bottleneck label used by the evaluator. Instead of only assigning a class label, each snapshot also stores an explanation and a list of key signals that document the intended diagnostic rationale. These fields support structured prompt design and provide a basis for future explanation-aware evaluation. The following code snippet shows the annotation field from an example snapshot labeled as `storage_bandwidth_saturation`.

```
"annotation": {
  "bottleneck": "storage_bandwidth_saturation",
  "confidence": "high",
  "explanation": "Write bandwidth reaches 95% of the storage. CPU usage(14%) shows
    cores are idle waiting on I/O.",
  "key_signals": [
    "bw_utilization_ratio: 0.95",
    "cpu_util_pct: 14",
    "io_time_ratio: 0.87",
    "avg_io_size_kb: 1024"
  ]
}
```

This dataset design provides a controlled benchmark for evaluating LLM-based diagnosis. Since the ground-truth labels and explanations are generated together with the metric patterns, the evaluation can directly compare model predictions with known bottleneck causes. At the same time, synthetic generation remains a limitation: the dataset cannot fully capture the noise, missing information, and transient behaviors of production HPC systems. We therefore view this dataset as a first-step benchmark for testing diagnostic reasoning under controlled conditions, while evaluation on real workflow traces remains an important direction for future work.

C. Baseline and Prompting Strategies

We evaluate five diagnosis strategies: a rule-based baseline, classify-only prompting, zero-shot prompting, few-shot prompting, and chain-of-thought prompting. These strategies differ in the amount of domain knowledge and reasoning guidance provided to the model, while using the same snapshot format and evaluation pipeline. For all LLM-based strategies, the ground-truth `annotation` field is removed before the snapshot is sent to the model.

1) *Rule-Based Baseline*: The rule-based baseline is a non-LLM method based on deterministic, priority-ordered threshold rules over the structured snapshot fields. It checks representative bottleneck signatures, such as high CPU utilization with low I/O time ratio for `compute_bound`, high network utilization for `network_io_bottleneck`, missing staging information with remote storage for `staging_inefficiency`, high bandwidth utilization for storage or read bandwidth saturation, small I/O size with high IOPS for

metadata_contention, and low parallelism for serialized_io. Since this baseline does not call an LLM, it has zero token cost and negligible latency.

This baseline provides a simple comparison point for judging whether LLM-based methods add value beyond fixed threshold logic. Its output follows the same general structure as the LLM outputs, including a predicted bottleneck, confidence, key signals, and explanation.

2) *Classify-Only Prompting*: Classify-only prompting is the simplest LLM-based strategy. The prompt provides the workflow snapshot and the list of possible bottleneck class names, but does not include class descriptions, examples, or reasoning instructions. The model is required to return only the predicted bottleneck field. In the implementation, this strategy uses a strict JSON schema to prevent the model from emitting extra fields. The following code snippet shows the simplified form of the prompt.

```
CLASSIFY_ONLY_PROMPT_TEMPLATE = """
Analyze the following HPC workflow execution snapshot and diagnose the primary
bottleneck.

Choose exactly one bottleneck class from this list:
- storage_bandwidth_saturation
- read_bandwidth_saturation
- metadata_contention
- lock_contention
- network_io_bottleneck
- serialized_io
- checkpointing_overhead
- compute_bound
- io_interference
- data_skew
- staging_inefficiency

Respond with a JSON object containing only the bottleneck field:
{{"bottleneck": "<one class from the list above>"}}

Snapshot:
{snapshot_json}
"""
```

This strategy tests whether class names alone are enough for the LLM to infer domain-specific I/O concepts. Because labels such as staging_inefficiency and metadata_contention require technical interpretation, classify-only prompting serves as a low-context LLM baseline.

3) *Zero-Shot Prompting*: Zero-shot prompting provides the model with explicit domain descriptions. The prompt includes all bottleneck classes together with their diagnostic signatures. For example, storage bandwidth saturation is described as write bandwidth saturating the storage device, metadata contention is described as many small file operations overwhelming the metadata server, and staging inefficiency is described as remote access caused by missing pre-staging. The model receives no labeled examples.

The expected output includes four fields: predicted bottleneck, confidence, key signals, and explanation. This matches our problem formulation, where the model is asked to return both a label and structured evidence for the prediction. The following code snippet shows the simplified response format used in the zero-shot prompt.

```
PROMPT_HEADER = """\n
Analyze the following HPC workflow execution snapshot and diagnose the primary
bottleneck.

Choose exactly one bottleneck class from this list:
- storage_bandwidth_saturation: write bandwidth saturates the storage device
- read_bandwidth_saturation: read bandwidth saturates the storage device
- metadata_contention: too many small file ops overwhelming the metadata server
- lock_contention: multiple tasks competing for write locks on shared files
- network_io_bottleneck: data movement over the network is the limiting factor
- serialized_io: I/O forced through a single writer/reader, preventing parallelism
- checkpointing_overhead: periodic checkpointing dominates stage time
- compute_bound: CPU computation is the bottleneck, I/O not a significant factor
- io_interference: external competing jobs degrade observed bandwidth
- data_skew: I/O load is unevenly distributed across nodes, creating hot spots
- staging_inefficiency: data was not pre-staged to a fast local tier before exec
"""\n

RESPONSE_FORMAT = """\n
Respond with a JSON object only no markdown, no extra text:
{{
  "bottleneck": "<one class from the list above>",
  "confidence": "<high | medium | low>",
  "key_signals": [<metric: value>, ...],
  "explanation": "<concise reasoning, 2-4 sentences>"
}}"""\n

USER_PROMPT_TEMPLATE = PROMPT_HEADER + "\n" + RESPONSE_FORMAT + """
```

This strategy tests whether domain descriptions and structured output instructions are sufficient for accurate diagnosis, without relying on demonstrations.

4) *Few-Shot Prompting*: Few-shot prompting extends the zero-shot prompt by adding five hand-picked demonstration examples. Each example consists of a snapshot with its ground-truth annotation, including the bottleneck label, key signals, and explanation. The examples are selected to cover commonly confused or representative cases, including read bandwidth saturation, lock contention, I/O interference, staging inefficiency, and metadata contention.

To avoid data leakage, the snapshots used as few-shot examples are excluded from the few-shot evaluation set. Compared with zero-shot prompting, this strategy provides concrete demonstrations of the desired mapping from snapshot signals to diagnosis, but also increases prompt length and token cost.

5) *Chain-of-Thought Prompting*: Chain-of-thought prompting adds an explicit diagnostic checklist to the zero-shot prompt. The prompt asks the model to reason through the snapshot before concluding,

following an ordered sequence of checks. This sequence covers compute versus I/O behavior, network bottlenecks, staging, interference, data skew, bandwidth saturation, access pattern, parallelism, checkpointing, and lock contention. The final response is still required to follow the same structured JSON format as the zero-shot prompt. The following code snippet shows the simplified checklist.

```
COT_PROMPT_TEMPLATE = PROMPT_HEADER + """
Reason step by step through the snapshot before concluding:
Step 1 - Compute vs I/O: compare cpu_util_pct and io_time_ratio.
Step 2 - Network: check network_util_ratio and storage type.
Step 3 - Staging: check data_staged and remote_access_latency_ms.
Step 4 - Interference: check competing_jobs.
Step 5 - Data skew: check io_imbalance_ratio and hot_node_count.
Step 6 - Bandwidth saturation: compare read/write bandwidth against peak.
Step 7 - Access pattern: check avg_io_size_kb and iops.
Step 8 - Parallelism: check whether I/O is serialized.
Step 9 - Checkpointing: check checkpoint_size_mb and num_checkpoints.
Step 10 - Lock contention: check shared files, parallelism, and bandwidth.
Step 11 - Conclude: state which single bottleneck class best fits the evidence.
""" + RESPONSE_FORMAT + """

Snapshot:
{snapshot_json}
"""
```

This strategy is designed to encourage systematic diagnosis and reduce missed evidence in boundary cases. It is expected to use more tokens than zero-shot prompting, but may improve reliability by making the diagnostic process more explicit.

Together, these strategies allow us to compare low-context prompting, domain-informed prompting, example-based prompting, and structured reasoning prompting under the same dataset and evaluation framework.

D. Evaluation Metrics

We evaluate each diagnosis strategy by comparing its predicted bottleneck label with the ground-truth annotation stored in each snapshot. The metrics are designed to answer the three research questions: accuracy and F1 measure diagnostic correctness, per-class metrics reveal which bottleneck types are easier or harder to identify, and token/latency measurements quantify the practical cost of each strategy. The evaluator also expects model outputs to be valid JSON objects so that all strategies can be parsed and compared under the same framework.

1) *Diagnosis Accuracy*: The primary metric is classification accuracy. For each snapshot, a prediction is considered correct if the predicted bottleneck label exactly matches the ground-truth bottleneck label. Given N evaluated snapshots, accuracy is defined as

$$\text{Accuracy} = \frac{N_{\text{correct}}}{N}.$$

Accuracy provides a direct measure of overall diagnostic success. It is especially useful for comparing prompting strategies at a high level, but it does not show whether errors are concentrated in specific bottleneck classes.

2) *Per-Class Performance*: To better understand class-level behavior, we compute precision, recall, and F1 score for each bottleneck class. For a class c , precision measures how often predictions of class c are correct, while recall measures how often true instances of class c are successfully detected:

$$P_c = \frac{TP_c}{TP_c + FP_c} \quad R_c = \frac{TP_c}{TP_c + FN_c}.$$

The per-class F1 score combines precision and recall and finally Macro-F1:

$$F1_c = \frac{2P_cR_c}{P_c + R_c} \quad \text{Macro-F1} = \frac{1}{|\mathcal{Y}|} \sum_{c \in \mathcal{Y}} F1_c$$

We then report macro F1, which averages F1 equally across all bottleneck classes:

$$\text{Macro-F1} = \frac{1}{|\mathcal{Y}|} \sum_{c \in \mathcal{Y}} F1_c$$

Macro F1 is important because it treats each bottleneck category equally, regardless of its frequency in the dataset. This helps identify whether a strategy performs consistently across classes instead of only performing well on easier or more frequent categories.

3) *Cost and Efficiency*: In addition to diagnostic correctness, we measure the computational cost of each strategy. For LLM-based strategies, the evaluator records prompt tokens, completion tokens, total tokens, and average token usage per snapshot. These values reflect the API cost and prompt length of each strategy. For example, few-shot and chain-of-thought prompts are expected to use more tokens than classify-only or zero-shot prompts because they include demonstrations or explicit diagnostic instructions.

We also measure latency using the duration of each model call, the total API time, and the average duration per snapshot. These timing metrics indicate whether a strategy is practical for interactive or repeated diagnosis. Finally, the evaluation script records auxiliary system statistics, including CPU usage and memory consumption during the run. Together, these metrics allow us to compare not only which

strategy is most accurate, but also which strategy provides the best trade-off between diagnostic quality and computational cost.

V. RESULTS

A. Overall Diagnosis Performance

Table II summarizes the overall diagnosis performance of all evaluated strategies. We evaluate two LLMs, GPT-4.1-mini and Claude Haiku 4.5, on the 500-snapshot dataset with 11 bottleneck classes. The rule-based baseline is model-independent, but it is listed under both model groups for easier comparison with the corresponding LLM strategies. For few-shot prompting, the evaluation set contains 495 snapshots because the five demonstration examples are excluded to avoid data leakage.

TABLE II
OVERALL DIAGNOSIS PERFORMANCE ACROSS MODELS AND STRATEGIES. TOKEN USAGE IS REPORTED AS A PROXY FOR API COST.

Model	Strategy	Total	Correct	Acc.	Macro-F1	Avg. Tokens
GPT-4.1-mini	Rule-based	500	410	0.820	0.827	0.0
	Classify-only	500	242	0.484	0.421	555.5
	Zero-shot	500	485	0.970	0.969	1054.4
	Few-shot	495	474	0.958	0.955	3672.6
	Chain-of-thought	500	484	0.968	0.965	1365.1
Claude Haiku 4.5	Rule-based	500	410	0.820	0.827	0.0
	Classify-only	500	246	0.492	0.458	796.9
	Zero-shot	500	479	0.958	0.956	1385.4
	Few-shot	495	486	0.982	0.981	4632.5
	Chain-of-thought	500	467	0.934	0.932	1778.2

The results on the workflow-informed synthetic dataset show a clear gap between low-context prompting and domain-informed prompting. Classify-only prompting performs poorly for both models, achieving 0.484 accuracy with GPT-4.1-mini and 0.492 accuracy with Claude Haiku 4.5. In contrast, zero-shot prompting substantially improves performance by adding bottleneck descriptions and diagnostic signals. GPT-4.1-mini with zero-shot prompting achieves 0.970 accuracy and 0.969 macro-F1, while Claude Haiku 4.5 with zero-shot prompting achieves 0.958 accuracy and 0.956 macro-F1.

The strongest overall result is achieved by Claude Haiku 4.5 with few-shot prompting, which reaches 0.982 accuracy and 0.981 macro-F1 on the 495 evaluated snapshots. However, this strategy also has the largest token usage, with an average of 4632.5 tokens per snapshot. For GPT-4.1-mini, few-shot prompting does not improve over zero-shot prompting: zero-shot achieves slightly higher accuracy and

macro-F1 while using substantially fewer tokens. Chain-of-thought prompting performs competitively for GPT-4.1-mini, but it is not the best strategy for Claude Haiku 4.5. These results suggest that domain descriptions are essential in this diagnostic setting, while additional examples or explicit reasoning steps do not uniformly improve performance across models.

B. Accuracy by LLM Confidence Tier

For each diagnosis, the LLM self-reports a confidence level (high, medium, or low) as part of its structured JSON output. This section examines whether that self-assessed confidence is a reliable signal of correctness.

A first notable observation is the complete absence of low-confidence predictions across both models and all strategies. Neither GPT-4.1-mini nor Claude Haiku 4.5 ever assigned a *low* confidence rating to any of the 500 diagnoses. This suggests that the models are systematically over-confident in their outputs, regardless of the actual difficulty of the snapshot (known tendency of instruction-tuned LLMs).

Despite the binary nature of the observed confidence distribution, the results consistently support the intuitive expectation: higher self-reported confidence correlates with higher accuracy. Across both models, predictions labeled high confidence are more accurate than those labeled medium confidence, for every prompting strategy where both tiers are represented.

This pattern holds for both models as shown in Figures 3 and 4. For Claude Haiku 4.5, the zero-shot strategy achieves 96.4% accuracy on high-confidence predictions versus 90.0% on medium-confidence ones. The few-shot strategy reaches a perfect 100.0% on medium-confidence predictions, suggesting that the additional context from examples helps the model both reason correctly *and* calibrate its uncertainty more reliably.

The most striking confidence gap appears in the *Classify-Only* strategy, where the model receives only class names with no descriptions. For GPT-4.1-mini, accuracy drops from 70.0% (medium confidence) to 46.0%. This inversion reveals a calibration failure specific to this strategy: when the model has no semantic grounding for the class names, its confidence signal becomes unreliable and even misleading. Same goes for Claude Haiku 4.5 under the same *Classify-Only* strategy. This shared miscalibration across both models under *Classify-Only* reinforces the interpretation that confidence is only a meaningful signal when the model has sufficient context to reason about the problem. Without class descriptions, the model essentially guesses, and its expressed confidence carries no diagnostic value.

These results suggest that LLM confidence scores can serve as a useful proxy for reliability in well-specified prompting regimes (zero-shot, few-shot, chain-of-thought), but should not be trusted when the model lacks domain grounding. In a production HPC monitoring system, confidence thresholds could be used to flag uncertain diagnoses for human review — but only if the prompting strategy provides sufficient context.

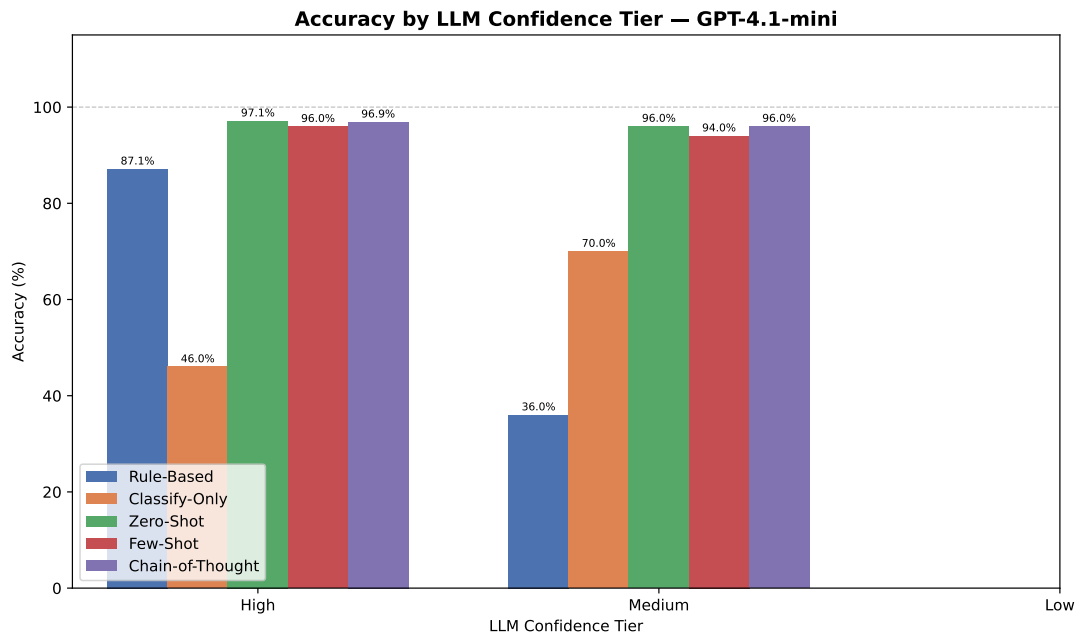


Fig. 3. Accuracy by confidence tier for GPT-4.1-mini.

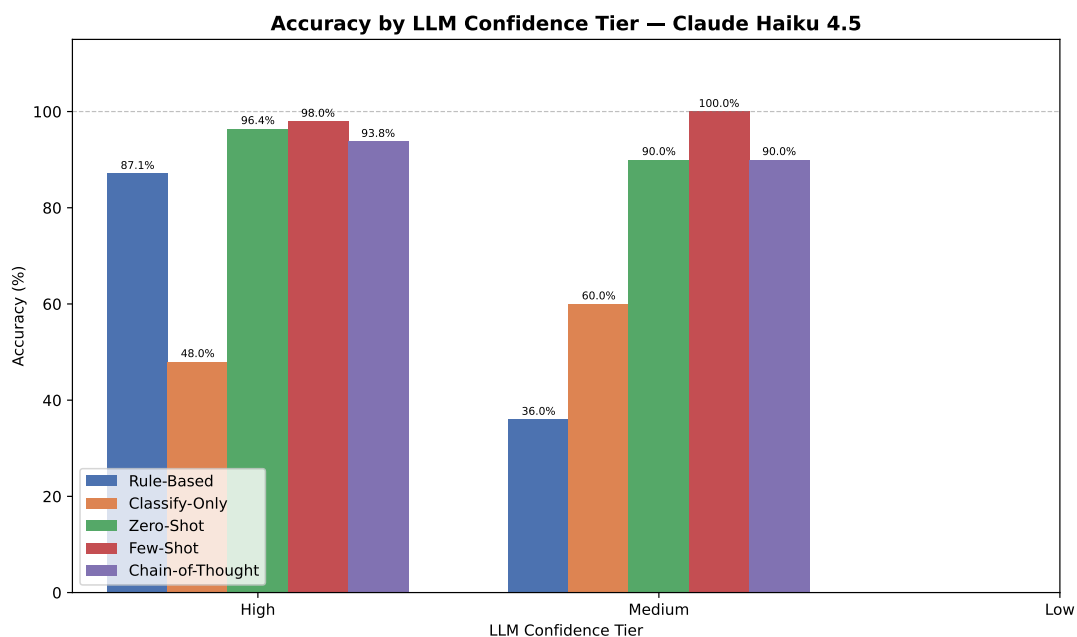


Fig. 4. Accuracy by confidence tier for Claude Haiku 4.5.

Cost–Accuracy Tradeoff

We use token usage as a proxy for API cost because LLM-based diagnosis is charged and constrained primarily by the number of input and output tokens. Figure 5 shows the tradeoff between average total tokens per snapshot and macro-F1. In this plot, a better strategy appears closer to the upper-left region, meaning higher diagnostic quality with lower token usage.

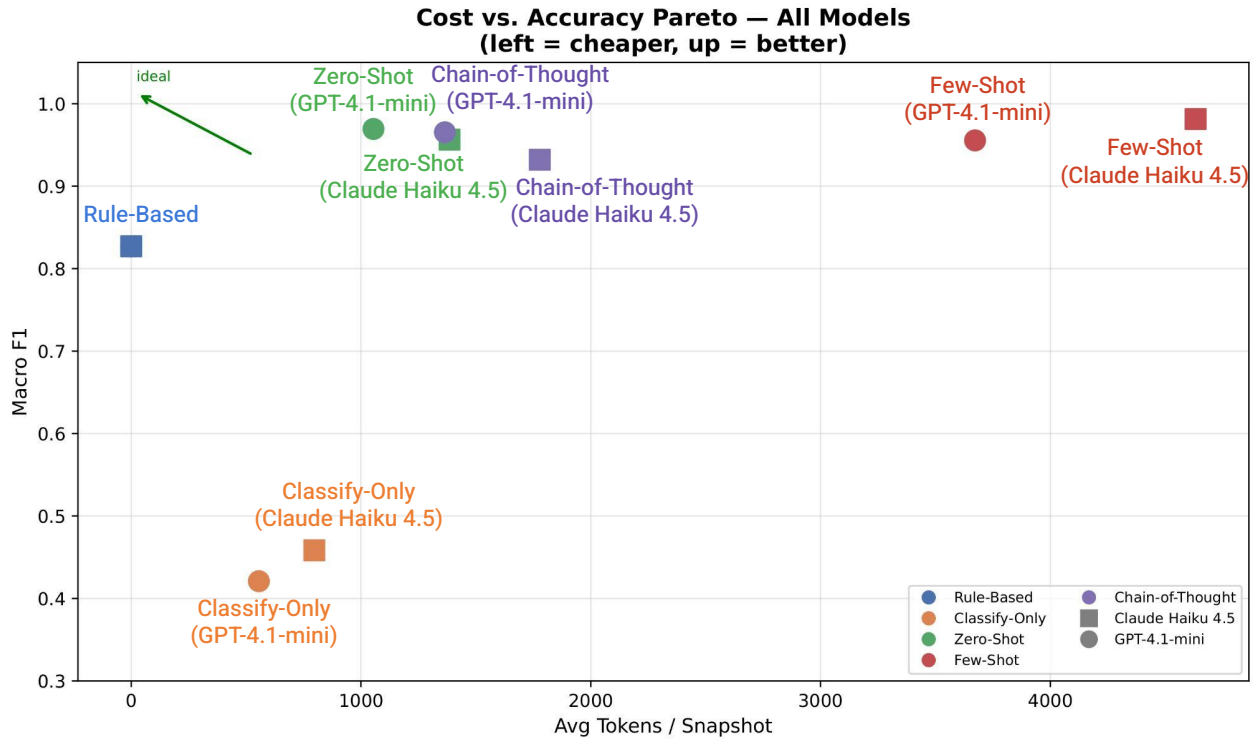


Fig. 5. Cost–accuracy tradeoff using average token usage per snapshot as a proxy for API cost.

The rule-based baseline has zero token usage and therefore represents the lowest-cost point, but its macro-F1 is lower than the strongest LLM-based methods. Classify-only prompting uses relatively few tokens compared with the other LLM strategies, but its macro-F1 is also much lower. This shows that short prompts alone are not sufficient for reliable bottleneck diagnosis when the model is not given domain descriptions or diagnostic guidance.

Zero-shot prompting provides the best cost-effective balance for GPT-4.1-mini. It achieves the highest GPT-4.1-mini macro-F1 of 0.969 while using only 1054.4 tokens per snapshot on average. Chain-of-thought prompting is close in accuracy for GPT-4.1-mini, with 0.965 macro-F1, but it requires more tokens on average. Few-shot prompting is much more expensive for GPT-4.1-mini and does not improve the final score over zero-shot prompting.

For Claude Haiku 4.5, few-shot prompting achieves the highest macro-F1 of 0.981, but it also has the largest token usage, with 4632.5 tokens per snapshot on average. Zero-shot prompting is less accurate than few-shot prompting for Claude Haiku 4.5, but it uses far fewer tokens and still reaches a strong macro-F1 of 0.956. Overall, the tradeoff shows that zero-shot prompting is the most efficient high-performing strategy, while few-shot prompting can provide the best accuracy for Claude Haiku 4.5 at substantially higher token cost.

C. Latency

In addition to token usage, latency is another practical factor for LLM-assisted diagnosis. We measure latency using the API response duration recorded for each snapshot. Since the rule-based baseline does not call an LLM, its runtime is negligible compared with the API-based strategies. For LLM-based methods, latency depends on both the prompt length and the length of the generated response.

Table III summarizes the total API time and average latency per snapshot for each model-strategy pair. The average latency is computed as

$$\text{Avg. Latency} = \frac{\text{API Time}}{\text{Total Evaluated Snapshots}}.$$

For few-shot prompting, the denominator is 495 because the five demonstration examples are excluded from evaluation.

TABLE III
API LATENCY SUMMARY ACROSS MODELS AND STRATEGIES.

Model	Strategy	API Time (s)	Avg. Latency (s)
GPT-4.1-mini	Rule-based	0.0	0.000
	Classify-only	520.7	1.041
	Zero-shot	1527.4	3.055
	Few-shot	1692.5	3.419
	Chain-of-thought	1829.2	3.658
Claude Haiku 4.5	Rule-based	0.0	0.000
	Classify-only	616.9	1.234
	Zero-shot	1600.4	3.201
	Few-shot	2557.7	5.167
	Chain-of-thought	1714.3	3.429

The latency trend is consistent with prompt complexity. Classify-only prompting has the lowest LLM latency because it uses the shortest prompt and returns only the bottleneck label. Zero-shot prompting

takes longer because it includes class descriptions and asks for key signals and explanations. Few-shot prompting has the highest latency for Claude Haiku 4.5, mainly because each request includes demonstration examples. Chain-of-thought prompting also increases latency because the model is instructed to follow a diagnostic checklist before returning the final structured response.

Figure 6 shows the latency distribution for chain-of-thought prompting across bottleneck classes. We use this strategy as the representative latency figure because it includes the most explicit diagnostic procedure among the prompting strategies and therefore reflects the runtime cost of structured reasoning. Since the latency figures for the other prompting strategies follow the same reporting format, we include them in the appendix for completeness.

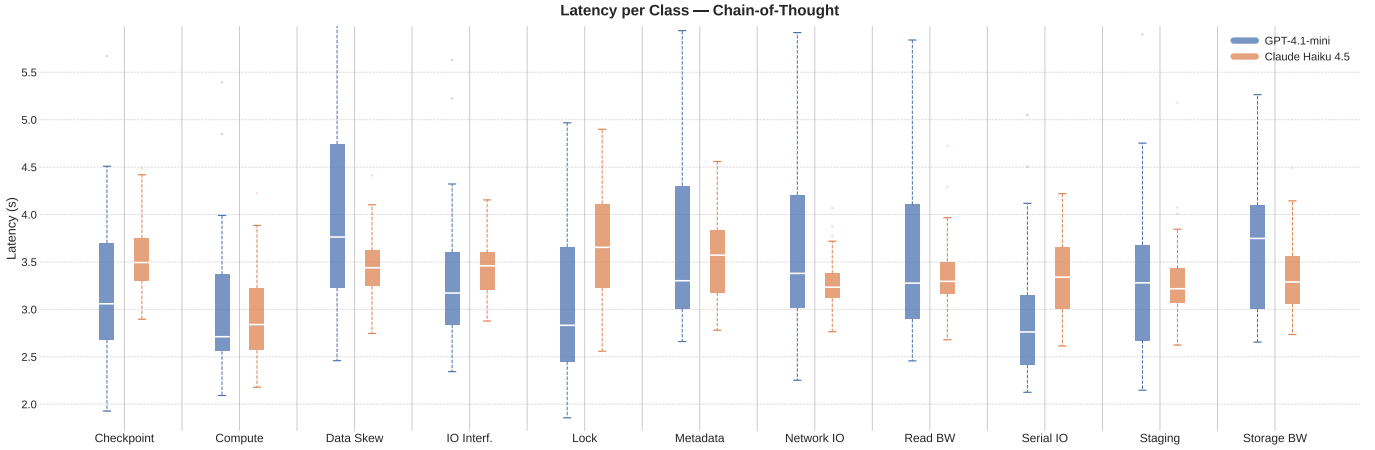


Fig. 6. Latency distribution for chain-of-thought prompting across bottleneck classes.

Across strategies, the main runtime cost comes from LLM API calls rather than local computation. The recorded CPU and memory usage remain small for the LLM-based runs, while API time dominates the wall-clock duration. Therefore, in this project, latency should be interpreted mainly as model response time rather than local processing overhead.

D. Per-class Confusion

Finally, we examine per-class performance using the confusion matrix of the best-performing model-strategy pair. Based on Table II, Claude Haiku 4.5 with few-shot prompting achieves the highest overall performance, with 0.982 accuracy and 0.981 macro-F1. Figure 7 shows its confusion matrix across the 11 bottleneck classes.

The confusion matrix shows strong diagonal performance for most bottleneck classes, which is consistent with the high macro-F1 score. This indicates that the model does not only perform well on the most

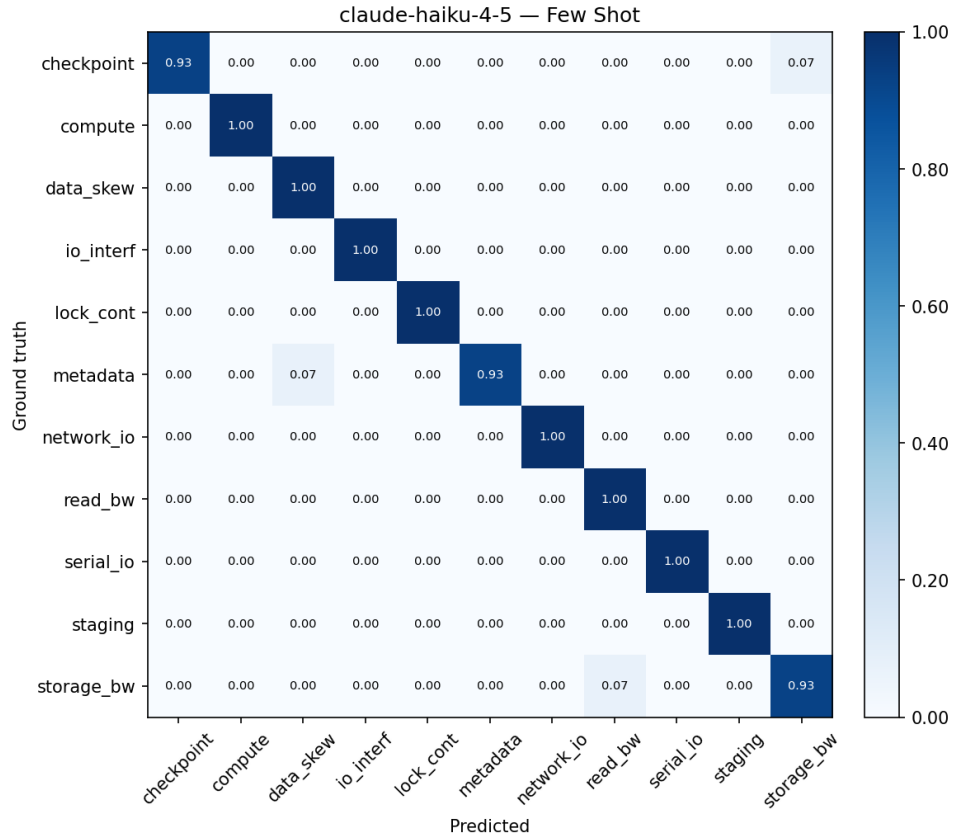


Fig. 7. Confusion matrix for Claude Haiku 4.5 with few-shot prompting.

frequent or easiest classes, but also maintains strong class-level performance across the label space. In particular, domain-informed prompting helps distinguish common I/O bottleneck patterns such as bandwidth saturation, network I/O bottlenecks, metadata contention, lock contention, serialized I/O, and compute-bound behavior.

The remaining errors are concentrated in a small number of class pairs where the diagnostic signals can overlap. For example, metadata contention and lock contention may both involve low bandwidth utilization and high I/O time, while checkpointing overhead and storage bandwidth saturation may both involve large write traffic. These confusions are expected because several bottleneck classes share similar surface-level metrics and require the model to identify the underlying cause from secondary signals such as access pattern, checkpoint frequency, shared-file behavior, and operation type.

Overall, the per-class results support the aggregate metrics reported earlier. The best-performing LLM configuration achieves high overall accuracy while preserving strong performance across individual bottleneck classes. The remaining confusion patterns motivate the more detailed error analysis in the next section.

VI. ANALYSIS AND DISCUSSION

A. Key Insights

The results reveal several practical lessons for applying LLMs to HPC workflow I/O bottleneck diagnosis under a controlled, workflow-informed setting. Overall, the most important finding is that LLM-based diagnosis depends less on the model seeing the raw snapshot alone and more on whether the prompt provides the right domain context, diagnostic categories, and expected reasoning structure.

1) Domain Context Drives Diagnosis Accuracy: The main insight is that domain context is essential for reliable diagnosis. Classify-only prompting gives the model the workflow snapshot and the candidate label names, but it does not explain what each bottleneck means or how it should be recognized. As a result, both models perform poorly in this setting. In contrast, zero-shot prompting provides class descriptions and diagnostic signatures, which leads to a large improvement in macro-F1 for both models.

This suggests that HPC I/O diagnosis is not a simple label-matching task. Labels such as `metadata_contention`, `lock_contention`, and `staging_inefficiency` require the model to interpret system-level evidence, not just recognize terminology. The prompt therefore acts as a form of domain knowledge injection: it defines the meaning of each bottleneck class and tells the model which metrics are relevant for distinguishing them. This is the central reason why domain-informed prompting substantially outperforms classify-only prompting in this setting.

2) Prompt Enhancements Are Model-Dependent: Additional prompt structure does not always lead to better accuracy. Few-shot prompting gives concrete examples of the expected mapping from snapshot signals to bottleneck labels. This helps Claude Haiku 4.5, which achieves the best overall performance with few-shot prompting. However, few-shot prompting does not improve GPT-4.1-mini over zero-shot prompting. This indicates that examples can help one model align with the task, but they can also provide limited or even unnecessary additional guidance when the class descriptions are already precise.

Chain-of-thought prompting shows a similar pattern. Its diagnostic checklist mirrors how an HPC performance analyst might inspect a workflow snapshot: first checking compute behavior, then bandwidth, network, staging, contention, skew, serialization, and checkpointing signals. This structure is useful because it encourages systematic inspection and makes the diagnostic procedure more explicit in the generated output. However, it is not uniformly the most accurate strategy. Therefore, few-shot examples and chain-of-thought reasoning should be viewed as model-dependent tuning options rather than guaranteed improvements over a well-designed zero-shot prompt.

3) *Practical Deployment Requires Accuracy–Cost Balance*: From a practical perspective, zero-shot prompting provides the strongest default tradeoff. It achieves high diagnostic performance for both models while using far fewer tokens than few-shot prompting. Few-shot prompting can be worthwhile when maximum accuracy is the priority, especially for Claude Haiku 4.5, but its higher token usage and latency make it less attractive as a default strategy. Classify-only prompting is cheaper but not reliable enough, while the rule-based baseline is free and fast but less accurate than the strongest LLM-based methods.

The remaining errors also show why deployment requires more than surface-level metric checks. Several bottleneck classes share similar symptoms, such as high I/O time, low CPU utilization, and moderate bandwidth utilization. For example, checkpointing overhead and storage bandwidth saturation can both involve large write traffic, while metadata contention and lock contention can both show low effective bandwidth. Correct diagnosis often depends on secondary signals such as checkpoint frequency, shared-file behavior, I/O parallelism, access size, network utilization, and data staging status. These hard cases suggest that the most useful future systems may combine structured rules, domain-informed prompts, and real workflow traces to improve robustness.

B. Limitations

1) *Dataset scale*: The final dataset contains 500 workflow snapshots, which is sufficient for comparing multiple prompting strategies and models in a course-project setting. At the same time, a larger dataset would provide stronger evidence for generalization. Future extensions could include more workflow families, more stages, and more diverse bottleneck combinations.

2) *Label-only evaluation*: Our quantitative evaluation mainly focuses on whether the predicted bottleneck label matches the ground truth. The LLM outputs also include confidence values, key signals, and explanations, which are useful for interpretation, but we do not separately score explanation quality in this project. A more complete evaluation could further assess whether the generated explanations are faithful, complete, and useful for HPC users.

C. Future Work

1) *Broader model coverage*: This project evaluates two LLMs, GPT-4.1-mini and Claude Haiku 4.5. A broader study could include additional closed-source and open-source models with different sizes, costs, and reasoning capabilities. This would help determine whether the observed differences among zero-shot, few-shot, and chain-of-thought prompting are model-specific or more general.

2) *Explanation-aware evaluation*: Another direction is to evaluate the quality of the generated explanations. In this project, the quantitative metrics primarily focus on bottleneck label correctness. Future work could design automatic metrics or human evaluation protocols to assess whether the explanations are useful for HPC applications.

VII. CONCLUSION

This project studied whether large language models can assist with I/O bottleneck diagnosis in HPC workflows from structured execution snapshots. We constructed a workflow-informed benchmark, implemented a reproducible diagnosis pipeline, and compared five strategies across GPT-4.1-mini and Claude Haiku 4.5. The results show that domain-informed prompting is the key factor for accurate diagnosis: classify-only prompting is not sufficient, while prompts that provide bottleneck descriptions and diagnostic signals achieve much stronger performance. Among the evaluated strategies, zero-shot prompting provides the strongest practical default, while few-shot prompting and chain-of-thought prompting can be useful but remain model-dependent.

Overall, this study suggests that LLMs can serve as useful diagnostic assistants when they are paired with structured workflow data and clear domain context. Rather than replacing traditional profiling tools or expert analysis, LLM-based diagnosis can help organize complex I/O signals into structured diagnostic outputs that are easier for users to inspect. At the same time, the current study is a first-step evaluation based on workflow-informed synthetic data. Future work should validate the approach on more realistic workflow traces, compare a broader set of models, and evaluate explanation quality in addition to label accuracy.

ACKNOWLEDGMENT

The authors thank the PhD Lead Meng Tang for her guidance and support throughout the project, and Teaching Assistant Keith Bateman and Professor Xian-He Sun for their feedback and instruction in the course.

REFERENCES

- [1] Peter Harrington et al. “Diagnosing parallel i/o bottlenecks in hpc applications”. In: *International Conference for High Performance Computing Networking Storage and Analysis (SCI7) ACM Student Research Competition (SRC)*. 2017, p. 4.
- [2] Jakob Lüttgau et al. “Toward Understanding I/O Behavior in HPC Workflows”. In: *2018 IEEE/ACM 3rd International Workshop on Parallel Data Storage & Data Intensive Scalable Computing Systems (PDSW-DISCS)*. 2018, pp. 64–75. DOI: 10.1109/PDSW-DISCS.2018.00012.
- [3] Luanzheng Guo et al. “Improving I/O-aware Workflow Scheduling via Data Flow Characterization and trade-off Analysis”. In: *2024 IEEE International Conference on Big Data (BigData)*. 2024, pp. 3674–3681. DOI: 10.1109/BigData62323.2024.10825855.
- [4] Yongxin Zhao et al. “When LLMs Listen to Experts: Accurate Failure Diagnosis in Operating Systems”. In: ().
- [5] Yusuke Miyashita, Patrick Kin Man Tung, and Johan Barthélemy. “LLM as HPC Expert: Extending RAG Architecture for HPC Data”. In: *arXiv preprint arXiv:2501.14733* (2024).
- [6] Dibyanayan Bandyopadhyay, Soham Bhattacharjee, and Asif Ekbal. “Thinking machines: A survey of llm based reasoning strategies”. In: *arXiv preprint arXiv:2503.10814* (2025).
- [7] Shane Snyder et al. “Modular HPC I/O Characterization with Darshan”. In: *2016 5th Workshop on Extreme-Scale Programming Tools (ESPT)*. 2016, pp. 9–17. DOI: 10.1109/ESPT.2016.006.
- [8] Alok Singh et al. “A machine learning approach for modular workflow performance prediction”. In: *Proceedings of the 12th workshop on workflows in support of large-scale science*. 2017, pp. 1–11.
- [9] Xin Huang, Ting Zhang, and Wen Zhao. “LogRules: Enhancing Log Analysis Capability of Large Language Models through Rules”. In: *Findings of the Association for Computational Linguistics: NAACL 2025*. 2025, pp. 452–470.
- [10] Runchu Tian et al. “Debugbench: Evaluating debugging capability of large language models”. In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024, pp. 4173–4198.
- [11] Meng Tang. *SPM (Storage Performance Matching) Analysis System*. <https://github.com/candiceT233/dpm>. 2025.

APPENDIX A

ADDITIONAL LATENCY RESULTS

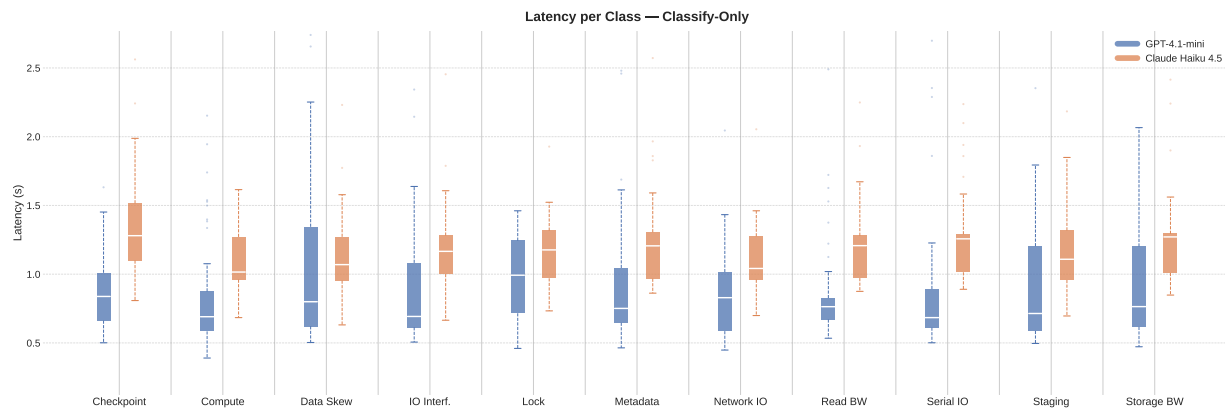


Fig. 8. Latency distribution for classify-only prompting across bottleneck classes.

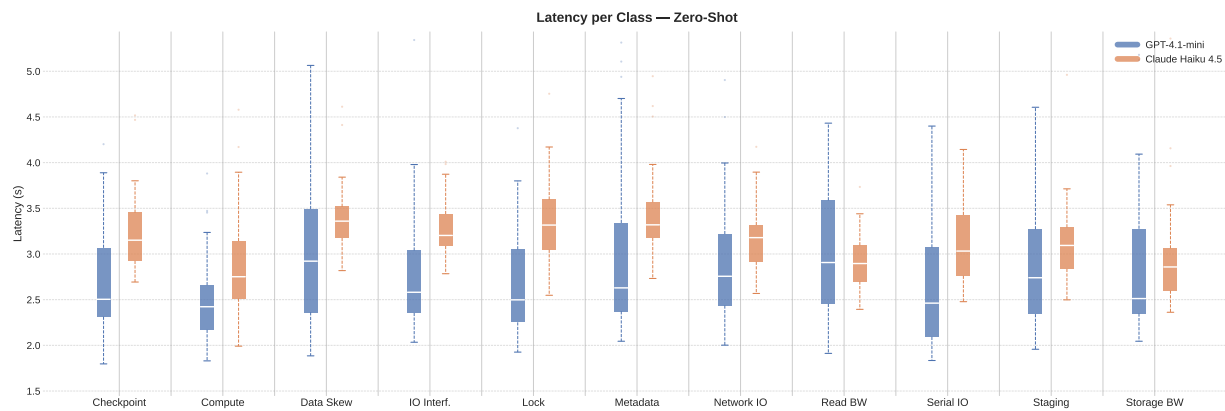


Fig. 9. Latency distribution for zero-shot prompting across bottleneck classes.

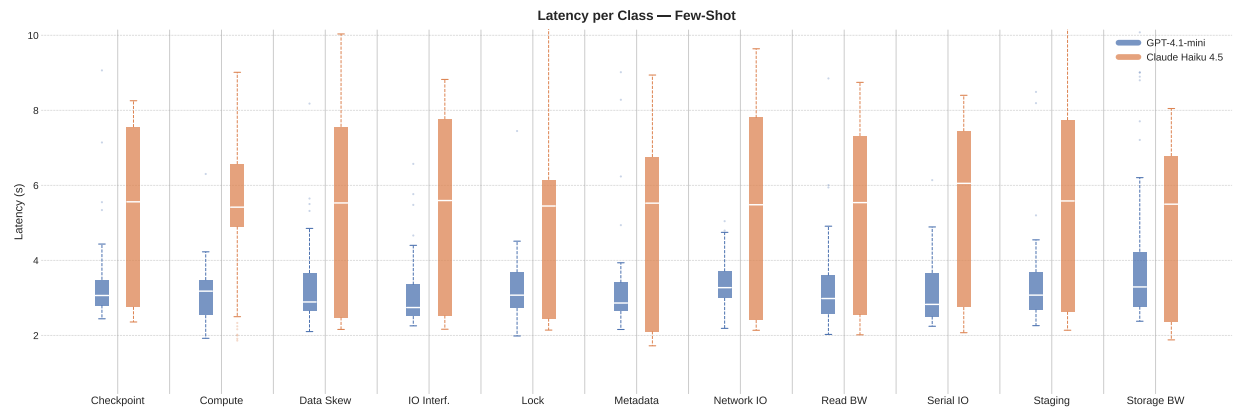


Fig. 10. Latency distribution for few-shot prompting across bottleneck classes.

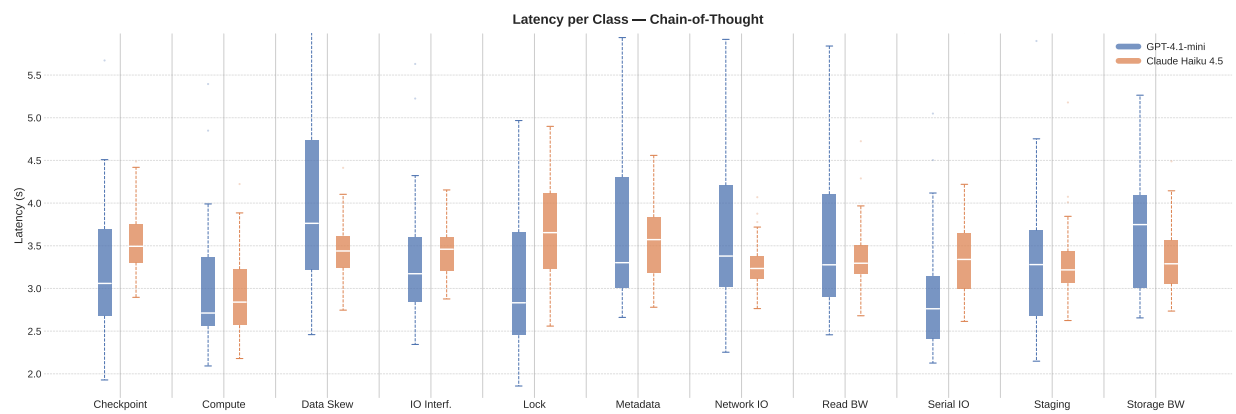


Fig. 11. Latency distribution for chain-of-thought prompting across bottleneck classes.

APPENDIX B

ADDITIONAL CONFUSION MATRICES

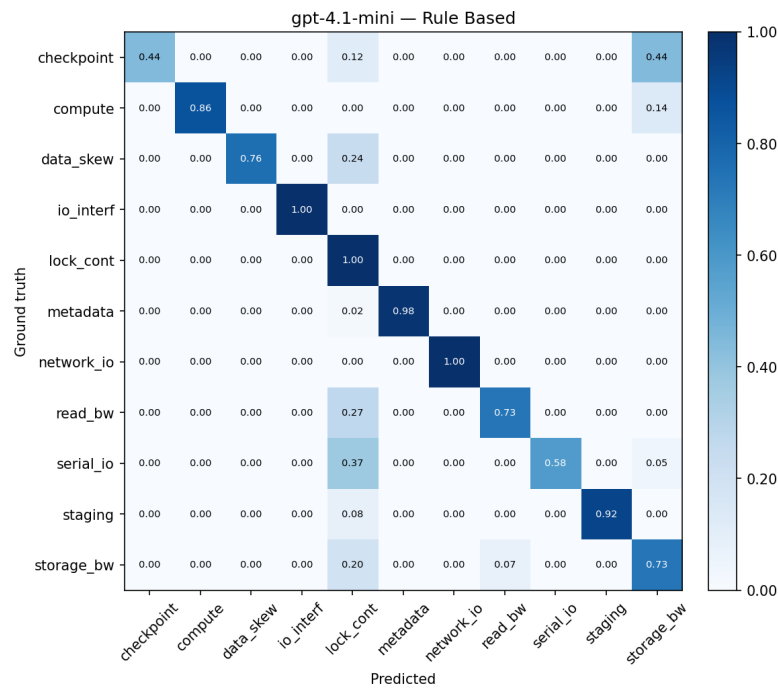


Fig. 12. Confusion matrix for the rule-based baseline.

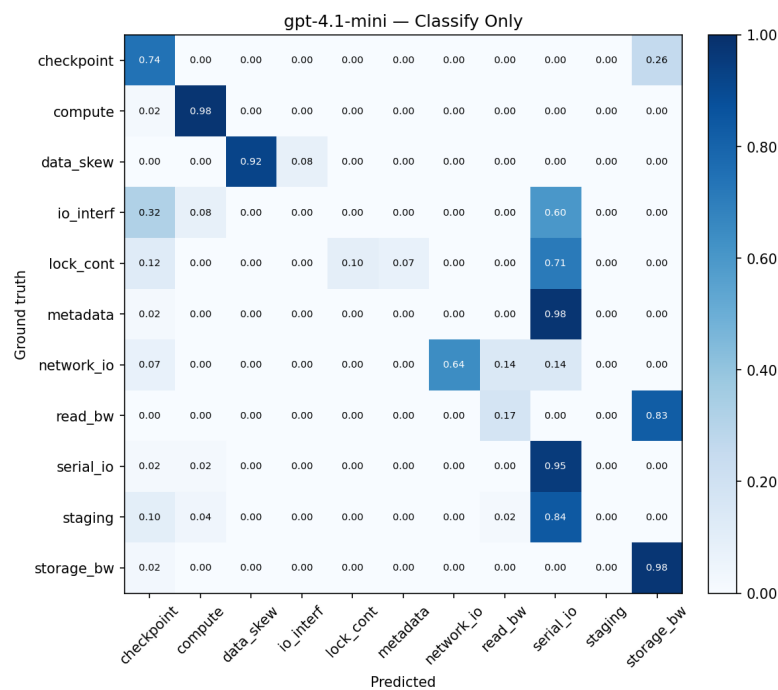


Fig. 13. Confusion matrix for GPT-4.1-mini with classify-only prompting.

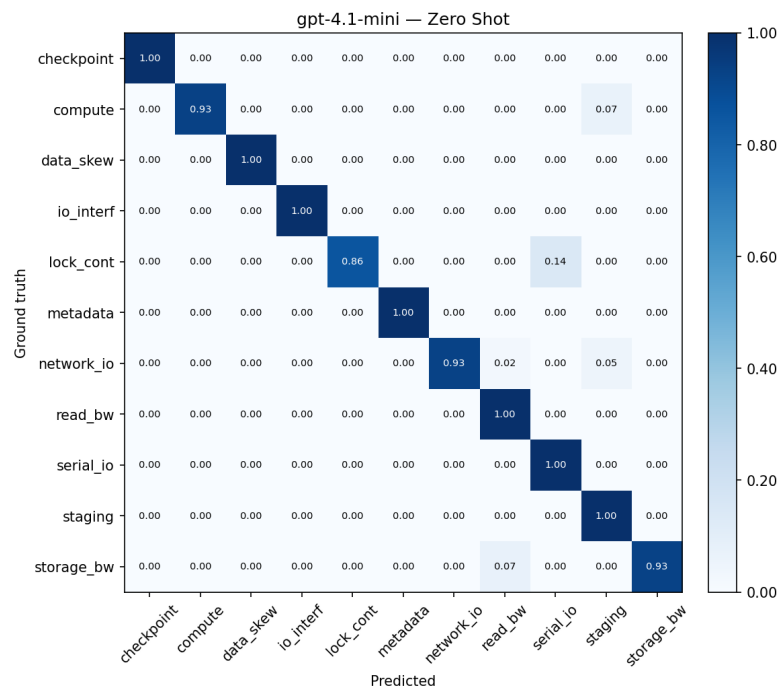


Fig. 14. Confusion matrix for GPT-4.1-mini with zero-shot prompting.

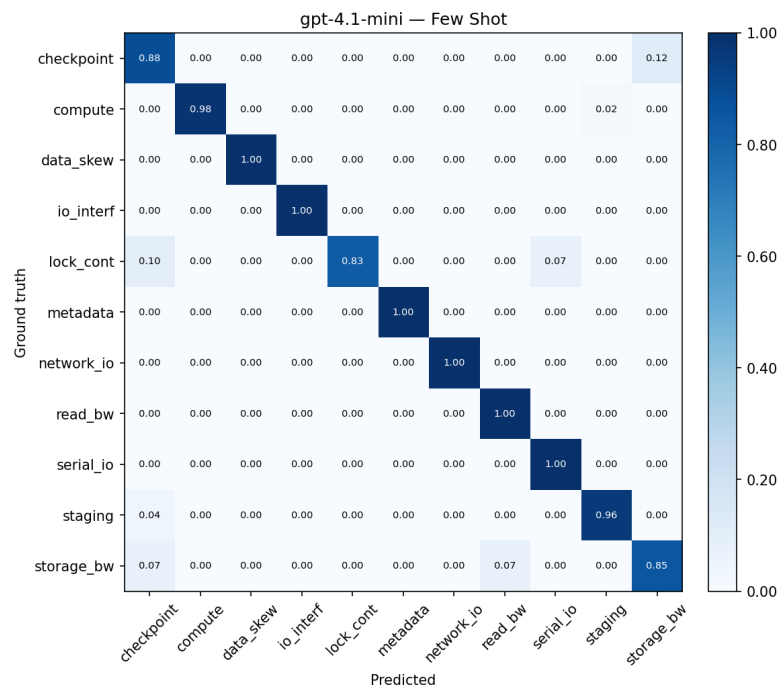


Fig. 15. Confusion matrix for GPT-4.1-mini with few-shot prompting.

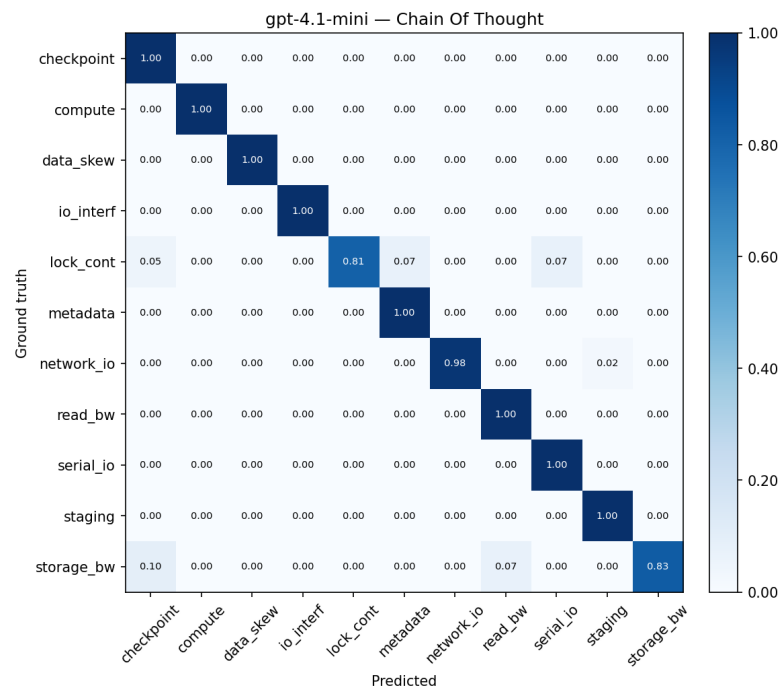


Fig. 16. Confusion matrix for GPT-4.1-mini with chain-of-thought prompting.

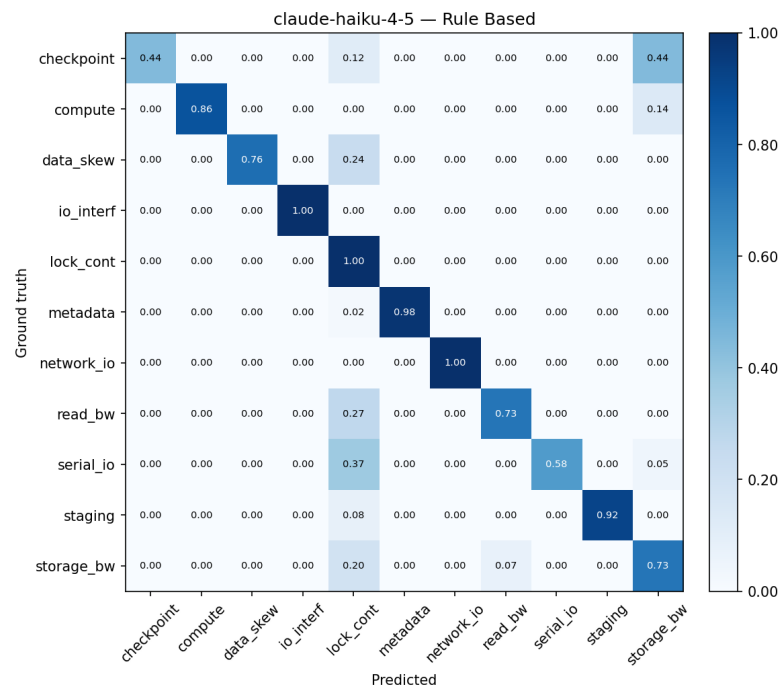


Fig. 17. Confusion matrix for the rule-based baseline.

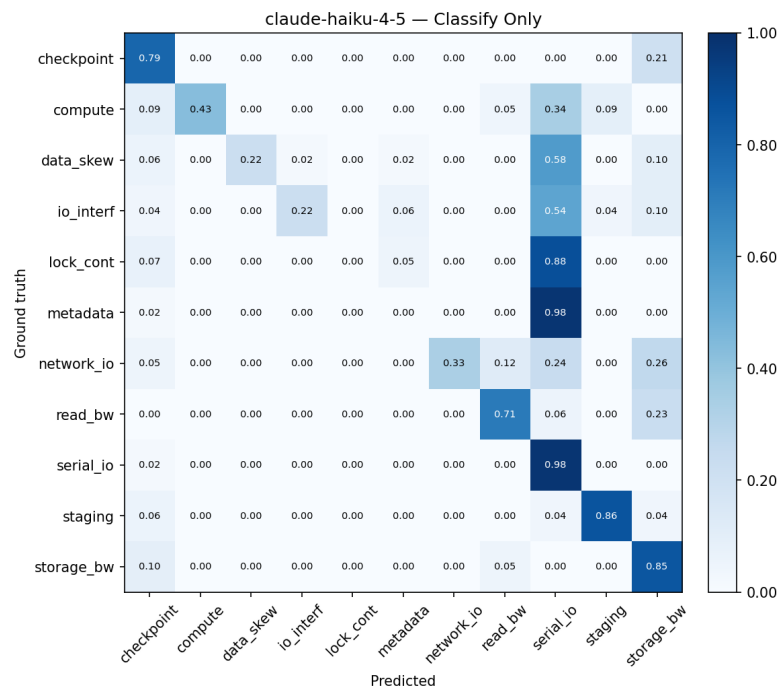


Fig. 18. Confusion matrix for Claude Haiku 4.5 with classify-only prompting.

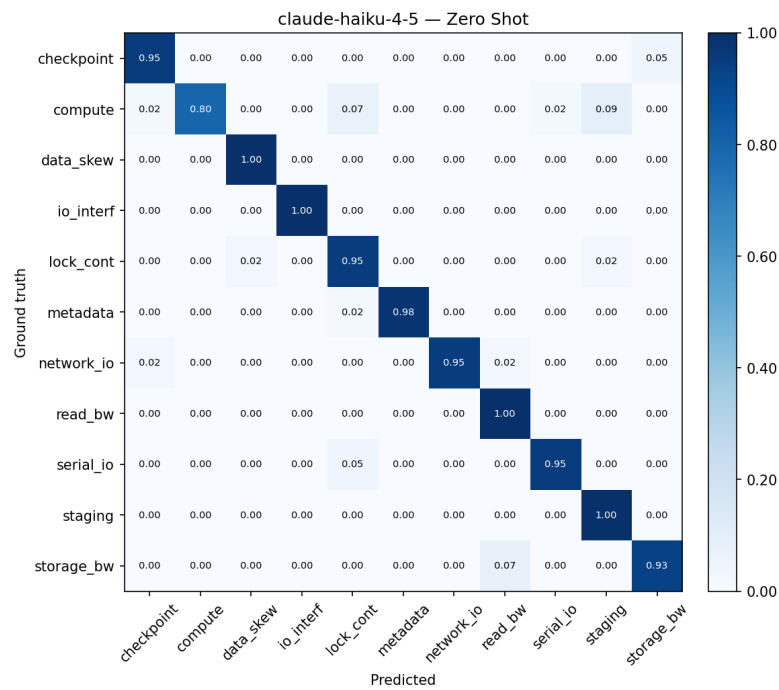


Fig. 19. Confusion matrix for Claude Haiku 4.5 with zero-shot prompting.

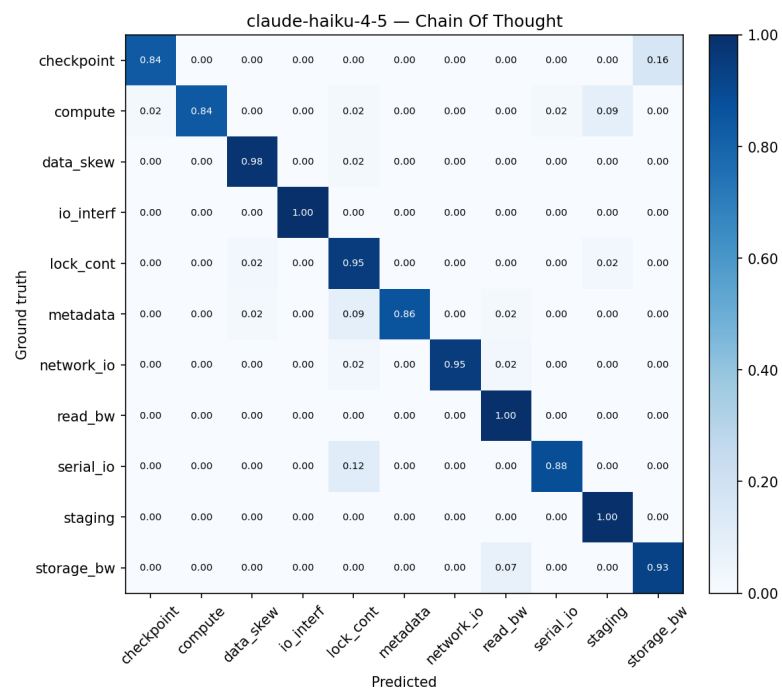


Fig. 20. Confusion matrix for Claude Haiku 4.5 with chain-of-thought prompting.